

Merge Sort and Asymptotic Analysis of Functions

Dr. Bugra Caskurlu¹

¹ School of Computing
New Uzbekistan University

August 27, 2025

Algorithm Design

Techniques

- You can choose from a wide range of algorithm design techniques. Insertion sort uses the **incremental** method: for each element $A[i]$, insert it into its proper place in the subarray $A[1 : i]$, having already sorted the subarray $A[1 : i - 1]$.
- What about the selection sort?

Algorithm Design

Techniques

- You can choose from a wide range of algorithm design techniques. Insertion sort uses the **incremental** method: for each element $A[i]$, insert it into its proper place in the subarray $A[1 : i]$, having already sorted the subarray $A[1 : i - 1]$.
- What about the selection sort? It does follow an incremental approach. At iteration k , the smallest k elements are placed into the first k spots in the array.

Algorithm Design

Techniques

- You can choose from a wide range of algorithm design techniques. Insertion sort uses the **incremental** method: for each element $A[i]$, insert it into its proper place in the subarray $A[1 : i]$, having already sorted the subarray $A[1 : i - 1]$.
- What about the selection sort? It does follow an incremental approach. At iteration k , the smallest k elements are placed into the first k spots in the array.
- Today, we will see an alternative design technique, named as **divide and conquer**. The **merge-sort algorithm** uses this design technique.

Algorithm Design

Techniques

- You can choose from a wide range of algorithm design techniques. Insertion sort uses the **incremental** method: for each element $A[i]$, insert it into its proper place in the subarray $A[1 : i]$, having already sorted the subarray $A[1 : i - 1]$.
- What about the selection sort? It does follow an incremental approach. At iteration k , the smallest k elements are placed into the first k spots in the array.
- Today, we will see an alternative design technique, named as **divide and conquer**. The **merge-sort algorithm** uses this design technique.
- One advantage of using an algorithm that follows the divide-and-conquer method is that analyzing its running time is often straightforward.

Divide and Conquer

Details

Algorithms using the divide and conquer paradigm are **recursive** in structure. The divide-and-conquer paradigm involves three steps at each level of the recursion:

- **Divide** the problem into a number of subproblems.
- **Conquer** the subproblems by solving them recursively. If the problem sizes are small enough, however, just solve the subproblems in a straightforward manner.
- **Combine** the solutions to the subproblems into the solution for the original problem.

Divide and Conquer

Details

Algorithms using the divide and conquer paradigm are **recursive** in structure. The divide-and-conquer paradigm involves three steps at each level of the recursion:

- **Divide** the problem into a number of subproblems.
- **Conquer** the subproblems by solving them recursively. If the problem sizes are small enough, however, just solve the subproblems in a straightforward manner.
- **Combine** the solutions to the subproblems into the solution for the original problem.

Merge Sort

In the **merge-sort** algorithm: We **divide** the n -element sequence to be sorted into two subsequences of $\frac{n}{2}$ elements each, we then **conquer**(sort) the two subsequences recursively using merge sort, and finally we **combine**(merge) the two sorted subsequences to produce the sorted answer.

Merge Sort

```
MERGE-SORT(A, p, r)
```

```
  if p < r
```

```
    q = (p + r) / 2;
```

```
    MERGE-SORT(A, p, q);
```

```
    MERGE-SORT(A, q + 1, r);
```

```
    MERGE(A, p, q, r);
```

Merge

```
MERGE(A, p, q, r)
```

```
n1 = q - p + 1; n2 = r - q;
```

```
create arrays L[1 .. n1 + 1] and R[1 .. n2 + 1];
```

```
for i = 1 to n1
```

```
    L[i] = A[p + i - 1];
```

```
for j = 1 to n2
```

```
    R[j] = A[q + j];
```

```
L[n1 + 1] = R[n2 + 1] = infinity
```

```
i = j = 1;
```

```
for k = p to r
```

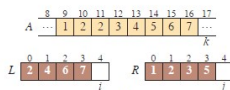
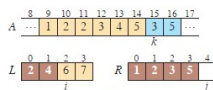
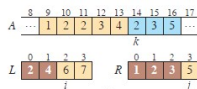
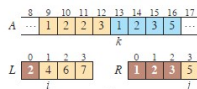
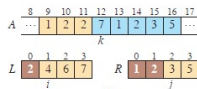
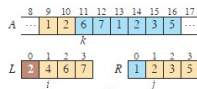
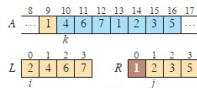
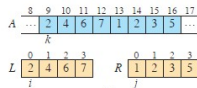
```
    if L[i] <= R[j]
```

```
        A[k] = L[i]; i++;
```

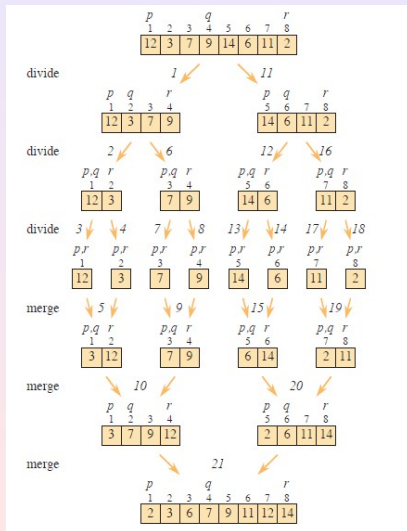
```
    else
```

```
        A[k] = R[j]; j++;
```

Operation of the Merge Procedure



Operation of the Merge Sort



Proof of Correctness

Loop Invariant

At the start of each iteration of the for loop, the subarray $A[p \dots k - 1]$ contains the $k - p$ smallest elements of $L[1 \dots n_1 + 1]$ and $R[1 \dots n_2 + 1]$, in sorted order. Moreover, $L[i]$ and $R[j]$ are the smallest elements of their arrays that have not been copied back into A .

Proof of Correctness

Loop Invariant

At the start of each iteration of the for loop, the subarray $A[p \dots k - 1]$ contains the $k - p$ smallest elements of $L[1 \dots n_1 + 1]$ and $R[1 \dots n_2 + 1]$, in sorted order. Moreover, $L[i]$ and $R[j]$ are the smallest elements of their arrays that have not been copied back into A .

Proving the Invariant

- **Initialization:** Prior to the first iteration of the loop, we have $k = p$, so that $A[p \dots k - 1]$ is empty. This empty array contains the $k - p = 0$ smallest elements of L and R , and $L[1]$ and $R[1]$ are the smallest elements of their arrays that have not been copied back into A .

Proof of Correctness

Loop Invariant

At the start of each iteration of the for loop, the subarray $A[p \dots k - 1]$ contains the $k - p$ smallest elements of $L[1 \dots n_1 + 1]$ and $R[1 \dots n_2 + 1]$, in sorted order. Moreover, $L[i]$ and $R[j]$ are the smallest elements of their arrays that have not been copied back into A .

Proving the Invariant

- **Initialization:** Prior to the first iteration of the loop, we have $k = p$, so that $A[p \dots k - 1]$ is empty. This empty array contains the $k - p = 0$ smallest elements of L and R , and $L[1]$ and $R[1]$ are the smallest elements of their arrays that have not been copied back into A .
- **Maintenance:** Suppose $L[i] \leq R[j]$. Then $L[i]$ is the smallest element not yet copied back into A . Because $A[p \dots k - 1]$ contains the $k - p$ smallest elements, at the end of the iteration, the subarray $A[p \dots k]$ contains the smallest $k - p + 1$ elements. Incrementing k and i reestablishes the loop invariant for the next iteration. Similar arguments hold if $L[i] > R[j]$.

Proof of Correctness

Loop Invariant

At the start of each iteration of the for loop, the subarray $A[p \dots k - 1]$ contains the $k - p$ smallest elements of $L[1 \dots n_1 + 1]$ and $R[1 \dots n_2 + 1]$, in sorted order. Moreover, $L[i]$ and $R[j]$ are the smallest elements of their arrays that have not been copied back into A .

Proving the Invariant

- **Initialization:** Prior to the first iteration of the loop, we have $k = p$, so that $A[p \dots k - 1]$ is empty. This empty array contains the $k - p = 0$ smallest elements of L and R , and $L[1]$ and $R[1]$ are the smallest elements of their arrays that have not been copied back into A .
- **Maintenance:** Suppose $L[i] \leq R[j]$. Then $L[i]$ is the smallest element not yet copied back into A . Because $A[p \dots k - 1]$ contains the $k - p$ smallest elements, at the end of the iteration, the subarray $A[p \dots k]$ contains the smallest $k - p + 1$ elements. Incrementing k and i reestablishes the loop invariant for the next iteration. Similar arguments hold if $L[i] > R[j]$.
- **Termination:** At termination, $k = r + 1$. By the loop invariant, the subarray $A[p \dots k - 1] = A[p \dots r]$ contains the $r - p + 1$ smallest elements (all elements of L and R except the sentinels) in sorted order.

Analysis of Merge Sort

Asymptotic Analysis of MERGE

- Notice that the portions of the pseudocode outside the for loops take constant time. Additionally, the block statement in the body of each loop takes constant time as well.

Analysis of Merge Sort

Asymptotic Analysis of MERGE

- Notice that the portions of the pseudocode outside the for loops take constant time. Additionally, the block statement in the body of each loop takes constant time as well.
- The first two loops take $\Theta(n_1 + n_2) = \Theta(n)$ -time. And there are n iterations of the last for loop and thus the time it takes to execute the last for loop is $\Theta(n)$.

Analysis of Merge Sort

Asymptotic Analysis of MERGE

- Notice that the portions of the pseudocode outside the for loops take constant time. Additionally, the block statement in the body of each loop takes constant time as well.
- The first two loops take $\Theta(n_1 + n_2) = \Theta(n)$ -time. And there are n iterations of the last for loop and thus the time it takes to execute the last for loop is $\Theta(n)$.
- Thus, the time required by the merge procedure is $\Theta(1) + \Theta(n) + \Theta(n) = \Theta(n)$.

Analysis of Merge Sort

Asymptotic Analysis of MERGE

- Notice that the portions of the pseudocode outside the for loops take constant time. Additionally, the block statement in the body of each loop takes constant time as well.
- The first two loops take $\Theta(n_1 + n_2) = \Theta(n)$ -time. And there are n iterations of the last for loop and thus the time it takes to execute the last for loop is $\Theta(n)$.
- Thus, the time required by the merge procedure is $\Theta(1) + \Theta(n) + \Theta(n) = \Theta(n)$.
- The procedure **MERGE** is a helper method(function), and we are actually interested in the time required by the **MERGE-SORT** rather than **MERGE**.

Analysis of Merge Sort

Asymptotic Analysis of MERGE

- Notice that the portions of the pseudocode outside the for loops take constant time. Additionally, the block statement in the body of each loop takes constant time as well.
- The first two loops take $\Theta(n_1 + n_2) = \Theta(n)$ -time. And there are n iterations of the last for loop and thus the time it takes to execute the last for loop is $\Theta(n)$.
- Thus, the time required by the merge procedure is $\Theta(1) + \Theta(n) + \Theta(n) = \Theta(n)$.
- The procedure **MERGE** is a helper method(function), and we are actually interested in the time required by the **MERGE-SORT** rather than **MERGE**. Before proceeding let's see how we analyze **divide-and conquer** algorithms.

Analyzing Divide-and-Conquer

Recurrences

- When an algorithm contains a recursive call to itself, its running time can often be described by a **recurrence equation** or **recurrence**, which describes the overall running time of a problem of size n in terms of the running time of smaller inputs.

Analyzing Divide-and-Conquer

Recurrences

- When an algorithm contains a recursive call to itself, its running time can often be described by a **recurrence equation** or **recurrence**, which describes the overall running time of a problem of size n in terms of the running time of smaller inputs.
- We can then use mathematical tools to solve the recurrence and provide bounds on the performance of the algorithm.

Analyzing Divide-and-Conquer

Recurrences

- When an algorithm contains a recursive call to itself, its running time can often be described by a **recurrence equation** or **recurrence**, which describes the overall running time of a problem of size n in terms of the running time of smaller inputs.
- We can then use mathematical tools to solve the recurrence and provide bounds on the performance of the algorithm.
- A recurrence for the running time of a divide-and conquer algorithm is based on the three steps of the basic paradigm.

Analyzing Divide-and-Conquer

Recurrences

- When an algorithm contains a recursive call to itself, its running time can often be described by a **recurrence equation** or **recurrence**, which describes the overall running time of a problem of size n in terms of the running time of smaller inputs.
- We can then use mathematical tools to solve the recurrence and provide bounds on the performance of the algorithm.
- A recurrence for the running time of a divide-and conquer algorithm is based on the three steps of the basic paradigm.
- Let $T(n)$ be the running time on a problem of size n . If the problem size is small enough, the straightforward solution takes constant time, which we write as $\Theta(1)$.

Analyzing Divide-and-Conquer

Recurrences

- When an algorithm contains a recursive call to itself, its running time can often be described by a **recurrence equation** or **recurrence**, which describes the overall running time of a problem of size n in terms of the running time of smaller inputs.
- We can then use mathematical tools to solve the recurrence and provide bounds on the performance of the algorithm.
- A recurrence for the running time of a divide-and conquer algorithm is based on the three steps of the basic paradigm.
- Let $T(n)$ be the running time on a problem of size n . If the problem size is small enough, the straightforward solution takes constant time, which we write as $\Theta(1)$.
- Suppose our division of the problem yields a subproblems, each of which is $\frac{1}{b}$ the size of the original. If we take $D(n)$ time to divide the problem into subproblems, and $C(n)$ time to combine the solutions to the subproblems into the solution of the original problem, we get the recurrence

$$T(n) = \begin{cases} \Theta(1) & \text{if } n \leq c \\ a \cdot T(\frac{n}{b}) + D(n) + C(n) & \text{otherwise} \end{cases}$$

Forming the Recurrence

Merge-Sort Recurrence

- **Divide:** The divide step just computes the middle of the array, which takes constant time. Thus, $D(n) = \Theta(1)$.

Forming the Recurrence

Merge-Sort Recurrence

- **Divide:** The divide step just computes the middle of the array, which takes constant time. Thus, $D(n) = \Theta(1)$.
- **Conquer:** We recursively solve two subproblems, each of size $\frac{n}{2}$, which contributes $2 \cdot T\left(\frac{n}{2}\right)$ to the running time.

Forming the Recurrence

Merge-Sort Recurrence

- **Divide:** The divide step just computes the middle of the array, which takes constant time. Thus, $D(n) = \Theta(1)$.
- **Conquer:** We recursively solve two subproblems, each of size $\frac{n}{2}$, which contributes $2 \cdot T(\frac{n}{2})$ to the running time.
- **Combine:** The **MERGE** procedure on an n -element subarray takes time $\Theta(n)$, so $C(n) = \Theta(n)$.



Forming the Recurrence

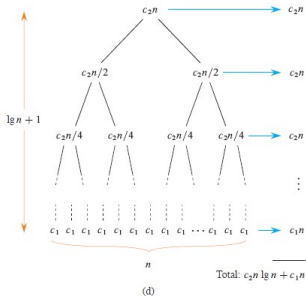
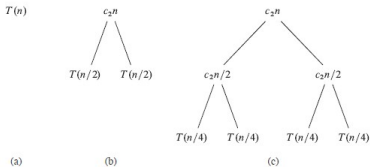
Merge-Sort Recurrence

- **Divide:** The divide step just computes the middle of the array, which takes constant time. Thus, $D(n) = \Theta(1)$.
- **Conquer:** We recursively solve two subproblems, each of size $\frac{n}{2}$, which contributes $2 \cdot T(\frac{n}{2})$ to the running time.
- **Combine:** The **MERGE** procedure on an n -element subarray takes time $\Theta(n)$, so $C(n) = \Theta(n)$.
- Thus, the running time of the **MERGE-SORT** algorithm can be described by the recurrence:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 2 \cdot T(\frac{n}{2}) + \Theta(n) & \text{otherwise} \end{cases}$$

- We will later see how to solve recurrences in detail but for now, we will say that the running time of this algorithm is $\Theta(n \lg n)$ as can be shown by drawing the recursion tree.

Solving the Merge-Sort Recurrence



Exercises

Exercise 1

Insertion-Sort can be expressed as a recursive procedure as follows: In order to sort $A[1 \dots n]$, we recursively sort $A[1 \dots n-1]$ and then insert $A[n]$ into the sorted array $A[1 \dots n-1]$. Write a recurrence for the running time of this recursive version of insertion sort.

Exercises

Exercise 1

Insertion-Sort can be expressed as a recursive procedure as follows: In order to sort $A[1 \dots n]$, we recursively sort $A[1 \dots n-1]$ and then insert $A[n]$ into the sorted array $A[1 \dots n-1]$. Write a recurrence for the running time of this recursive version of insertion sort. What would be the recurrence if the place $A[n]$ will be inserted is computed via binary search rather than the linear search?

Exercises

Exercise 1

Insertion-Sort can be expressed as a recursive procedure as follows: In order to sort $A[1 \dots n]$, we recursively sort $A[1 \dots n-1]$ and then insert $A[n]$ into the sorted array $A[1 \dots n-1]$. Write a recurrence for the running time of this recursive version of insertion sort. What would be the recurrence if the place $A[n]$ will be inserted is computed via binary search rather than the linear search?

Answer

The recurrence in both cases will be the following:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ T(n-1) + O(n) & \text{otherwise} \end{cases}$$

Exercises Cont.

Exercise 2

Although merge-sort runs in $\Theta(n \cdot \lg n)$ worst-case time and insertion sort runs in $\Theta(n^2)$ **worst-case** time, the constant factors in insertion sort make it faster for small n . Thus, it makes sense to use insertion sort within merge sort when subproblems become sufficiently small. Consider a modification to merge sort in which $\frac{n}{k}$ sublists of length k are sorted using insertion sort and then merged using the standard merging mechanism, where k is a value to be determined.

- Show that the $\frac{n}{k}$ sublists, each of length k , can be sorted by insertion sort in $\Theta(n \cdot k)$ worst-case time.
- Show that the sublists can be merged in $\Theta(n \lg(\frac{n}{k}))$ worst-case time.
- Given that the modified algorithm runs in $\Theta(n \cdot k + n \cdot \lg(\frac{n}{k}))$ **worst-case time**, what is the largest asymptotic (Θ -notation) value of k as a function of n for which the modified algorithm has the same asymptotic running time as standard merge sort?

Asymptotic Analysis

Why do we need Asymptotic Analysis?

- The order of growth of the running time of an algorithm gives a simple characterization of the algorithm's efficiency and also allows us to compare the relative performance of alternative algorithms.

Asymptotic Analysis

Why do we need Asymptotic Analysis?

- The order of growth of the running time of an algorithm gives a simple characterization of the algorithm's efficiency and also allows us to compare the relative performance of alternative algorithms.
- For example, merge sort with its $\Theta(n \cdot \lg n)$ running time beats insertions sort, whose **worst-case** running time is $\Theta(n^2)$, for all but very small inputs.

Asymptotic Analysis

Why do we need Asymptotic Analysis?

- The order of growth of the running time of an algorithm gives a simple characterization of the algorithm's efficiency and also allows us to compare the relative performance of alternative algorithms.
- For example, merge sort with its $\Theta(n \cdot \lg n)$ running time beats insertion sort, whose **worst-case** running time is $\Theta(n^2)$, for all but very small inputs.
- Although we can sometimes determine the exact running time of an algorithm, as we did for insertion sort, the extra precision is not usually worth the effort of computing it since the multiplicative constants and lower-order terms of an exact running time are dominated by the effects of the input size itself.

Asymptotic Analysis

Why do we need Asymptotic Analysis?

- The order of growth of the running time of an algorithm gives a simple characterization of the algorithm's efficiency and also allows us to compare the relative performance of alternative algorithms.
- For example, merge sort with its $\Theta(n \cdot \lg n)$ running time beats insertion sort, whose **worst-case** running time is $\Theta(n^2)$, for all but very small inputs.
- Although we can sometimes determine the exact running time of an algorithm, as we did for insertion sort, the extra precision is not usually worth the effort of computing it since the multiplicative constants and lower-order terms of an exact running time are dominated by the effects of the input size itself.
- In analysing an algorithm, whether through a worst-case or average-case analysis, we use asymptotic notation to describe the behavior of the algorithm on large instances of the problems.

Asymptotic Analysis

Why do we need Asymptotic Analysis?

- The order of growth of the running time of an algorithm gives a simple characterization of the algorithm's efficiency and also allows us to compare the relative performance of alternative algorithms.
- For example, merge sort with its $\Theta(n \cdot \lg n)$ running time beats insertion sort, whose **worst-case** running time is $\Theta(n^2)$, for all but very small inputs.
- Although we can sometimes determine the exact running time of an algorithm, as we did for insertion sort, the extra precision is not usually worth the effort of computing it since the multiplicative constants and lower-order terms of an exact running time are dominated by the effects of the input size itself.
- In analysing an algorithm, whether through a worst-case or average-case analysis, we use asymptotic notation to describe the behavior of the algorithm on large instances of the problems.
- While asymptotic analysis has its drawbacks (what if the asymptotic behavior appears only for extremely large instances that would never arise in practice?, and what if the constant factors and lower-order terms are extremely large for reasonable instance sizes?), it provides a clean characterization of at least one essential facet of the behavior of the algorithm.

Types of Asymptotic Analysis

Upper and Lower Bounding

- Asymptotic analysis aims at providing lower bound and upper bounds on the rate of growth of functions.

Types of Asymptotic Analysis

Upper and Lower Bounding

- Asymptotic analysis aims at providing lower bound and upper bounds on the rate of growth of functions.
- It accomplishes this by grouping functions with *similar* growth rate into families, thereby providing a framework within which a new function can be located and thus characterized.

Types of Asymptotic Analysis

Upper and Lower Bounding

- Asymptotic analysis aims at providing lower bound and upper bounds on the rate of growth of functions.
- It accomplishes this by grouping functions with *similar* growth rate into families, thereby providing a framework within which a new function can be located and thus characterized.
- We should distinguish two types of analysis: that which focuses on behavior exhibited *almost everywhere* (a.e.) and that which focuses on behavior exhibited *infinitely often* (i.o.).

Types of Asymptotic Analysis

Upper and Lower Bounding

- Asymptotic analysis aims at providing lower bound and upper bounds on the rate of growth of functions.
- It accomplishes this by grouping functions with *similar* growth rate into families, thereby providing a framework within which a new function can be located and thus characterized.
- We should distinguish two types of analysis: that which focuses on behavior exhibited *almost everywhere* (a.e.) and that which focuses on behavior exhibited *infinitely often* (i.o.).

a.e and i.o

- For functions of interest to us, which are mostly functions from $\mathbb{N} \rightarrow \mathbb{N}$, a.e. behavior is behavior that is observed on **all but a finite number of function arguments**;

Types of Asymptotic Analysis

Upper and Lower Bounding

- Asymptotic analysis aims at providing lower bound and upper bounds on the rate of growth of functions.
- It accomplishes this by grouping functions with *similar* growth rate into families, thereby providing a framework within which a new function can be located and thus characterized.
- We should distinguish two types of analysis: that which focuses on behavior exhibited *almost everywhere* (a.e.) and that which focuses on behavior exhibited *infinitely often* (i.o.).

a.e and i.o

- For functions of interest to us, which are mostly functions from $\mathbb{N} \rightarrow \mathbb{N}$, a.e. behavior is behavior that is observed on **all but a finite number of function arguments**; in consequence, there must exist some number $N \in \mathbb{N}$ such that the function exhibits that behavior for all arguments $n \geq N$.

Types of Asymptotic Analysis

Upper and Lower Bounding

- Asymptotic analysis aims at providing lower bound and upper bounds on the rate of growth of functions.
- It accomplishes this by grouping functions with *similar* growth rate into families, thereby providing a framework within which a new function can be located and thus characterized.
- We should distinguish two types of analysis: that which focuses on behavior exhibited *almost everywhere* (a.e.) and that which focuses on behavior exhibited *infinitely often* (i.o.).

a.e and i.o

- For functions of interest to us, which are mostly functions from $\mathbb{N} \rightarrow \mathbb{N}$, a.e. behavior is behavior that is observed on **all but a finite number of function arguments**; in consequence, there must exist some number $N \in \mathbb{N}$ such that the function exhibits that behavior for all arguments $n \geq N$.
- In contrast, i.o. behavior is observable on **an infinite number of arguments**;

Types of Asymptotic Analysis

Upper and Lower Bounding

- Asymptotic analysis aims at providing lower bound and upper bounds on the rate of growth of functions.
- It accomplishes this by grouping functions with *similar* growth rate into families, thereby providing a framework within which a new function can be located and thus characterized.
- We should distinguish two types of analysis: that which focuses on behavior exhibited *almost everywhere* (a.e.) and that which focuses on behavior exhibited *infinitely often* (i.o.).

a.e and i.o

- For functions of interest to us, which are mostly functions from $\mathbb{N} \rightarrow \mathbb{N}$, a.e. behavior is behavior that is observed on **all but a finite number of function arguments**; in consequence, there must exist some number $N \in \mathbb{N}$ such that the function exhibits that behavior for all arguments $n \geq N$.
- In contrast, i.o. behavior is observable on **an infinite number of arguments**; we can only state that, for each number $N \in \mathbb{N}$, there exists a larger number $N' \geq N$ such that the function exhibits the desired behavior on argument N' .

Which asymptotic analysis to use?

Asymptotic Analysis Traditions

- Traditional asymptotic analysis uses a.e. behavior, as does calculus defining the limit of a function when its argument grows unbounded.

Which asymptotic analysis to use?

Asymptotic Analysis Traditions

- Traditional asymptotic analysis uses a.e. behavior, as does calculus defining the limit of a function when its argument grows unbounded.
- Recall that $\lim_{n \rightarrow \infty} f(n) = a$ is defined by $\forall \varepsilon > 0, \exists N > 0$ such that $\forall n \geq N$ we have $|f(n) - a| \leq \varepsilon$.

Which asymptotic analysis to use?

Asymptotic Analysis Traditions

- Traditional asymptotic analysis uses a.e. behavior, as does calculus defining the limit of a function when its argument grows unbounded.
- Recall that $\lim_{n \rightarrow \infty} f(n) = a$ is defined by $\forall \varepsilon > 0, \exists N > 0$ such that $\forall n \geq N$ we have $|f(n) - a| \leq \varepsilon$.
- In other words, for all $\varepsilon > 0$, the value $|f(n) - a|$ is almost everywhere no larger than ε .

Which asymptotic analysis to use?

Asymptotic Analysis Traditions

- Traditional asymptotic analysis uses a.e. behavior, as does calculus defining the limit of a function when its argument grows unbounded.
- Recall that $\lim_{n \rightarrow \infty} f(n) = a$ is defined by $\forall \varepsilon > 0, \exists N > 0$ such that $\forall n \geq N$ we have $|f(n) - a| \leq \varepsilon$.
- In other words, for all $\varepsilon > 0$, the value $|f(n) - a|$ is almost everywhere no larger than ε .

Deriving Bounds

- We are using asymptotic analysis to derive *bounds* on the run-time (or space) of an algorithm.

Which asymptotic analysis to use?

Asymptotic Analysis Traditions

- Traditional asymptotic analysis uses a.e. behavior, as does calculus defining the limit of a function when its argument grows unbounded.
- Recall that $\lim_{n \rightarrow \infty} f(n) = a$ is defined by $\forall \varepsilon > 0, \exists N > 0$ such that $\forall n \geq N$ we have $|f(n) - a| \leq \varepsilon$.
- In other words, for all $\varepsilon > 0$, the value $|f(n) - a|$ is almost everywhere no larger than ε .

Deriving Bounds

- We are using asymptotic analysis to derive *bounds* on the run-time (or space) of an algorithm.
- **While a.e. analysis is justified for upper bounds, i.o. analysis is mostly a better choice for lower bounds.**

Which asymptotic analysis to use?

Asymptotic Analysis Traditions

- Traditional asymptotic analysis uses a.e. behavior, as does calculus defining the limit of a function when its argument grows unbounded.
- Recall that $\lim_{n \rightarrow \infty} f(n) = a$ is defined by $\forall \varepsilon > 0, \exists N > 0$ such that $\forall n \geq N$ we have $|f(n) - a| \leq \varepsilon$.
- In other words, for all $\varepsilon > 0$, the value $|f(n) - a|$ is almost everywhere no larger than ε .

Deriving Bounds

- We are using asymptotic analysis to derive *bounds* on the run-time (or space) of an algorithm.
- **While a.e. analysis is justified for upper bounds, i.o. analysis is mostly a better choice for lower bounds.**
- Since most of complexity theory is concerned with upper bounds, we will use the a.e. analysis in this class.

General Framework

Notation

- The notations we use to describe the asymptotic runtime of an algorithm are defined in terms of functions whose domains are the set of natural numbers $\mathbb{N} = \{0, 1, 2, \dots\}$.

General Framework

Notation

- The notations we use to describe the asymptotic runtime of an algorithm are defined in terms of functions whose domains are the set of natural numbers $\mathbb{N} = \{0, 1, 2, \dots\}$.
- Such notations are convenient for describing the worst case running-time function $T(n)$, which is usually defined only on integer input sizes.

General Framework

Notation

- The notations we use to describe the asymptotic runtime of an algorithm are defined in terms of functions whose domains are the set of natural numbers $\mathbb{N} = \{0, 1, 2, \dots\}$.
- Such notations are convenient for describing the worst case running-time function $T(n)$, which is usually defined only on integer input sizes.
- It is sometimes convenient to *abuse* asymptotic notation in a variety of ways, such as, extending the domain to real numbers, or restricting it to a subset of natural numbers.

General Framework

Notation

- The notations we use to describe the asymptotic runtime of an algorithm are defined in terms of functions whose domains are the set of natural numbers $\mathbb{N} = \{0, 1, 2, \dots\}$.
- Such notations are convenient for describing the worst case running-time function $T(n)$, which is usually defined only on integer input sizes.
- It is sometimes convenient to *abuse* asymptotic notation in a variety of ways, such as, extending the domain to real numbers, or restricting it to a subset of natural numbers.
- It is important to understand the precise meaning of the notation so that **we do not misuse when we are abusing the notation!**

Θ-Notation

Definition

For a given function $g(n)$, we denote by $\Theta(g(n))$ the **set of functions**:

$\Theta(g(n)) = \{f(n) \mid \text{there exist positive constants } c_1, c_2, \text{ and } n_0 \text{ such that } 0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \text{ for all } n \geq n_0\}.$

Θ-Notation

Definition

For a given function $g(n)$, we denote by $\Theta(g(n))$ the **set of functions**:

$\Theta(g(n)) = \{f(n) \mid \text{there exist positive constants } c_1, c_2, \text{ and } n_0 \text{ such that } 0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \text{ for all } n \geq n_0\}$.

Details

- A function $f(n)$ belongs to the set $\Theta(g(n))$ if there exists positive constants c_1 and c_2 such that it can be sandwiched between $c_1 \cdot g(n)$ and $c_2 \cdot g(n)$ for sufficiently large n .

Θ -Notation

Definition

For a given function $g(n)$, we denote by $\Theta(g(n))$ the **set of functions**:

$\Theta(g(n)) = \{f(n) \mid \text{there exist positive constants } c_1, c_2, \text{ and } n_0 \text{ such that } 0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \text{ for all } n \geq n_0\}$.

Details

- A function $f(n)$ belongs to the set $\Theta(g(n))$ if there exists positive constants c_1 and c_2 such that it can be sandwiched between $c_1 \cdot g(n)$ and $c_2 \cdot g(n)$ for sufficiently large n .
- We will usually write $f(n) = \Theta(g(n))$, and mean $f(n) \in \Theta(g(n))$.

Θ-Notation

Definition

For a given function $g(n)$, we denote by $\Theta(g(n))$ the **set of functions**:

$\Theta(g(n)) = \{f(n) \mid \text{there exist positive constants } c_1, c_2, \text{ and } n_0 \text{ such that } 0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \text{ for all } n \geq n_0\}.$

Details

- A function $f(n)$ belongs to the set $\Theta(g(n))$ if there exists positive constants c_1 and c_2 such that it can be sandwiched between $c_1 \cdot g(n)$ and $c_2 \cdot g(n)$ for sufficiently large n .
- We will usually write $f(n) = \Theta(g(n))$, and mean $f(n) \in \Theta(g(n))$.
- The definition of $\Theta(g(n))$ requires that every member $f(n) \in \Theta(g(n))$ be **asymptotically nonnegative**, that is, $f(n)$ is nonnegative whenever n is sufficiently large.

Θ -Notation

Definition

For a given function $g(n)$, we denote by $\Theta(g(n))$ the **set of functions**:

$\Theta(g(n)) = \{f(n) \mid \text{there exist positive constants } c_1, c_2, \text{ and } n_0 \text{ such that } 0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \text{ for all } n \geq n_0\}.$

Details

- A function $f(n)$ belongs to the set $\Theta(g(n))$ if there exists positive constants c_1 and c_2 such that it can be sandwiched between $c_1 \cdot g(n)$ and $c_2 \cdot g(n)$ for sufficiently large n .
- We will usually write $f(n) = \Theta(g(n))$, and mean $f(n) \in \Theta(g(n))$.
- The definition of $\Theta(g(n))$ requires that every member $f(n) \in \Theta(g(n))$ be **asymptotically nonnegative**, that is, $f(n)$ is nonnegative whenever n is sufficiently large.
- **Example:** $f(n) = \frac{1}{2} \cdot n^2 - 3 \cdot n$.

Θ-Notation

Definition

For a given function $g(n)$, we denote by $\Theta(g(n))$ the **set of functions**:

$\Theta(g(n)) = \{f(n) \mid \text{there exist positive constants } c_1, c_2, \text{ and } n_0 \text{ such that } 0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \text{ for all } n \geq n_0\}.$

Details

- A function $f(n)$ belongs to the set $\Theta(g(n))$ if there exists positive constants c_1 and c_2 such that it can be sandwiched between $c_1 \cdot g(n)$ and $c_2 \cdot g(n)$ for sufficiently large n .
- We will usually write $f(n) = \Theta(g(n))$, and mean $f(n) \in \Theta(g(n))$.
- The definition of $\Theta(g(n))$ requires that every member $f(n) \in \Theta(g(n))$ be **asymptotically nonnegative**, that is, $f(n)$ is nonnegative whenever n is sufficiently large.
- **Example:** $f(n) = \frac{1}{2} \cdot n^2 - 3 \cdot n$. Let $c_1 = \frac{1}{14}$, and $c_2 = \frac{1}{2}$. Then $0 \leq c_1 \cdot n^2 \leq f(n) \leq c_2 \cdot n^2$ for $n \geq 7$, and thus $f(n) = \Theta(n^2)$.

Θ-Notation

Definition

For a given function $g(n)$, we denote by $\Theta(g(n))$ the **set of functions**:

$\Theta(g(n)) = \{f(n) \mid \text{there exist positive constants } c_1, c_2, \text{ and } n_0 \text{ such that } 0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \text{ for all } n \geq n_0\}.$

Details

- A function $f(n)$ belongs to the set $\Theta(g(n))$ if there exists positive constants c_1 and c_2 such that it can be sandwiched between $c_1 \cdot g(n)$ and $c_2 \cdot g(n)$ for sufficiently large n .
- We will usually write $f(n) = \Theta(g(n))$, and mean $f(n) \in \Theta(g(n))$.
- The definition of $\Theta(g(n))$ requires that every member $f(n) \in \Theta(g(n))$ be **asymptotically nonnegative**, that is, $f(n)$ is nonnegative whenever n is sufficiently large.
- **Example:** $f(n) = \frac{1}{2} \cdot n^2 - 3 \cdot n$. Let $c_1 = \frac{1}{14}$, and $c_2 = \frac{1}{2}$. Then $0 \leq c_1 \cdot n^2 \leq f(n) \leq c_2 \cdot n^2$ for $n \geq 7$, and thus $f(n) = \Theta(n^2)$.
- **Example:** Show that $f(n) = 6 \cdot n^3 \neq \Theta(n^2)$.

O-Notation

Definition

For a given function $g(n)$, we denote by $O(g(n))$ the **set of functions**:

$O(g(n)) = \{f(n) \mid \text{there exist positive constants } c, \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0\}.$

O-Notation

Definition

For a given function $g(n)$, we denote by $O(g(n))$ the **set of functions**:

$O(g(n)) = \{f(n) \mid \text{there exist positive constants } c, \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0\}.$

Details

- The Θ -notation bounds a function from above and below. When we have only an asymptotic upper bound, we use O -notation.

O-Notation

Definition

For a given function $g(n)$, we denote by $O(g(n))$ the **set of functions**:

$O(g(n)) = \{f(n) \mid \text{there exist positive constants } c, \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0\}.$

Details

- The Θ -notation bounds a function from above and below. When we have only an asymptotic upper bound, we use O -notation.
- The running time of **INSERTION-SORT** is, for instance, $O(n^2)$, but not $\Theta(n^2)$.

O-Notation

Definition

For a given function $g(n)$, we denote by $O(g(n))$ the **set of functions**:

$O(g(n)) = \{f(n) \mid \text{there exist positive constants } c, \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0\}.$

Details

- The Θ -notation bounds a function from above and below. When we have only an asymptotic upper bound, we use O -notation.
- The running time of **INSERTION-SORT** is, for instance, $O(n^2)$, but not $\Theta(n^2)$.
- We write $f(n) = O(g(n))$ to indicate that $f(n) \in O(g(n))$.

O-Notation

Definition

For a given function $g(n)$, we denote by $O(g(n))$ the **set of functions**:

$O(g(n)) = \{f(n) \mid \text{there exist positive constants } c, \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0\}.$

Details

- The Θ -notation bounds a function from above and below. When we have only an asymptotic upper bound, we use O -notation.
- The running time of **INSERTION-SORT** is, for instance, $O(n^2)$, but not $\Theta(n^2)$.
- We write $f(n) = O(g(n))$ to indicate that $f(n) \in O(g(n))$.
- Notice that $\Theta(g(n)) \subseteq O(g(n))$ for any asymptotically nonnegative function $g(n)$.

Ω -Notation

Definition

For a given function $g(n)$, we denote by $\Omega(g(n))$ the **set of functions**:

$\Omega(g(n)) = \{f(n) \mid \text{there exist positive constants } c, \text{ and } n_0 \text{ such that } 0 \leq c \cdot g(n) \leq f(n) \text{ for all } n \geq n_0\}$.

Ω -Notation

Definition

For a given function $g(n)$, we denote by $\Omega(g(n))$ the **set of functions**:

$\Omega(g(n)) = \{f(n) \mid \text{there exist positive constants } c, \text{ and } n_0 \text{ such that } 0 \leq c \cdot g(n) \leq f(n) \text{ for all } n \geq n_0\}.$

Details

- Just as O -notation provides an upper bound on a function, Ω -notation provides an asymptotic lower bound.

Ω -Notation

Definition

For a given function $g(n)$, we denote by $\Omega(g(n))$ the **set of functions**:

$\Omega(g(n)) = \{f(n) \mid \text{there exist positive constants } c, \text{ and } n_0 \text{ such that } 0 \leq c \cdot g(n) \leq f(n) \text{ for all } n \geq n_0\}$.

Details

- Just as O -notation provides an upper bound on a function, Ω -notation provides an asymptotic lower bound.
- The running time of **INSERTION-SORT** is, for instance, $\Omega(n)$.

Ω -Notation

Definition

For a given function $g(n)$, we denote by $\Omega(g(n))$ the **set of functions**:

$\Omega(g(n)) = \{f(n) \mid \text{there exist positive constants } c, \text{ and } n_0 \text{ such that } 0 \leq c \cdot g(n) \leq f(n) \text{ for all } n \geq n_0\}$.

Details

- Just as O -notation provides an upper bound on a function, Ω -notation provides an asymptotic lower bound.
- The running time of **INSERTION-SORT** is, for instance, $\Omega(n)$.

Theorem

For any two functions $f(n)$ and $g(n)$, we have $f(n) = \Theta(g(n))$ if and only if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.

o-Notation

Definition

For a given function $g(n)$, we denote by $o(g(n))$ the **set of functions**:

$o(g(n)) = \{f(n) \mid \text{for any positive constant } c > 0, \text{ there exists a constant } n_0 > 0 \text{ such that } 0 \leq f(n) < c \cdot g(n) \text{ for all } n \geq n_0\}.$

o-Notation

Definition

For a given function $g(n)$, we denote by $o(g(n))$ the **set of functions**:

$o(g(n)) = \{f(n) \mid \text{for any positive constant } c > 0, \text{ there exists a constant } n_0 > 0 \text{ such that } 0 \leq f(n) < c \cdot g(n) \text{ for all } n \geq n_0\}.$

Details

- The definitions of O -notation and o -notation are similar. The main difference is that in $f(n) = O(g(n))$, the bound $0 \leq f(n) \leq c \cdot g(n)$ holds for *some* constant $c > 0$, but in $f(n) = o(g(n))$, the bound $0 \leq f(n) < c \cdot g(n)$ holds for *all* constants $c > 0$.

o-Notation

Definition

For a given function $g(n)$, we denote by $o(g(n))$ the **set of functions**:

$o(g(n)) = \{f(n) \mid \text{for any positive constant } c > 0, \text{ there exists a constant } n_0 > 0 \text{ such that } 0 \leq f(n) < c \cdot g(n) \text{ for all } n \geq n_0\}.$

Details

- The definitions of O -notation and o -notation are similar. The main difference is that in $f(n) = O(g(n))$, the bound $0 \leq f(n) \leq c \cdot g(n)$ holds for *some* constant $c > 0$, but in $f(n) = o(g(n))$, the bound $0 \leq f(n) < c \cdot g(n)$ holds for *all* constants $c > 0$.
- Intuitively, in the o -notation, the function $f(n)$ becomes insignificant relative to $g(n)$ as n approaches infinity; that is, $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$.

o-Notation

Definition

For a given function $g(n)$, we denote by $o(g(n))$ the **set of functions**:

$o(g(n)) = \{f(n) \mid \text{for any positive constant } c > 0, \text{ there exists a constant } n_0 > 0 \text{ such that } 0 \leq f(n) < c \cdot g(n) \text{ for all } n \geq n_0\}.$

Details

- The definitions of O -notation and o -notation are similar. The main difference is that in $f(n) = O(g(n))$, the bound $0 \leq f(n) \leq c \cdot g(n)$ holds for *some* constant $c > 0$, but in $f(n) = o(g(n))$, the bound $0 \leq f(n) < c \cdot g(n)$ holds for *all* constants $c > 0$.
- Intuitively, in the o -notation, the function $f(n)$ becomes insignificant relative to $g(n)$ as n approaches infinity; that is, $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$.
- $f(n) = o(g(n))$ means that the function $f(n)$ is asymptotically smaller than the function $g(n)$. This notation is useful in designing algorithms. If we set out to improve on an existing algorithm, we want that improvement to show in the subsequent asymptotic analysis, and thus we need notation that does not accept equality.

ω -Notation

Definition

For a given function $g(n)$, we denote by $\omega(g(n))$ the **set of functions**:

$\omega(g(n)) = \{f(n) \mid \text{for any positive constant } c > 0, \text{ there exists a constant } n_0 > 0 \text{ such that } 0 \leq c \cdot g(n) < f(n) \text{ for all } n \geq n_0\}.$

ω -Notation

Definition

For a given function $g(n)$, we denote by $\omega(g(n))$ the **set of functions**:

$\omega(g(n)) = \{f(n) \mid \text{for any positive constant } c > 0, \text{ there exists a constant } n_0 > 0 \text{ such that } 0 \leq c \cdot g(n) < f(n) \text{ for all } n \geq n_0\}$.

Details

- **Example:** $f(n) = \frac{n^2}{2} = \omega(n)$, but $f(n) \neq \omega(n^2)$.

ω -Notation

Definition

For a given function $g(n)$, we denote by $\omega(g(n))$ the **set of functions**:

$\omega(g(n)) = \{f(n) \mid \text{for any positive constant } c > 0, \text{ there exists a constant } n_0 > 0 \text{ such that } 0 \leq c \cdot g(n) < f(n) \text{ for all } n \geq n_0\}$.

Details

- **Example:** $f(n) = \frac{n^2}{2} = \omega(n)$, but $f(n) \neq \omega(n^2)$.
- Intuitively, in the ω -notation, the function $g(n)$ becomes insignificant relative to $f(n)$ as n approaches infinity; that is, $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$.

ω -Notation

Definition

For a given function $g(n)$, we denote by $\omega(g(n))$ the **set of functions**:

$\omega(g(n)) = \{f(n) \mid \text{for any positive constant } c > 0, \text{ there exists a constant } n_0 > 0 \text{ such that } 0 \leq c \cdot g(n) < f(n) \text{ for all } n \geq n_0\}$.

Details

- **Example:** $f(n) = \frac{n^2}{2} = \omega(n)$, but $f(n) \neq \omega(n^2)$.
- Intuitively, in the ω -notation, the function $g(n)$ becomes insignificant relative to $f(n)$ as n approaches infinity; that is, $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$.

Relational Properties

Transitivity

- $f(n) = \Theta(g(n))$ and $g(n) = \Theta(h(n))$ implies $f(n) = \Theta(h(n))$.
- $f(n) = O(g(n))$ and $g(n) = O(h(n))$ implies $f(n) = O(h(n))$.
- $f(n) = \Omega(g(n))$ and $g(n) = \Omega(h(n))$ implies $f(n) = \Omega(h(n))$.
- $f(n) = o(g(n))$ and $g(n) = o(h(n))$ implies $f(n) = o(h(n))$.
- $f(n) = \omega(g(n))$ and $g(n) = \omega(h(n))$ implies $f(n) = \omega(h(n))$.

Relational Properties

Transitivity

- $f(n) = \Theta(g(n))$ and $g(n) = \Theta(h(n))$ implies $f(n) = \Theta(h(n))$.
- $f(n) = O(g(n))$ and $g(n) = O(h(n))$ implies $f(n) = O(h(n))$.
- $f(n) = \Omega(g(n))$ and $g(n) = \Omega(h(n))$ implies $f(n) = \Omega(h(n))$.
- $f(n) = o(g(n))$ and $g(n) = o(h(n))$ implies $f(n) = o(h(n))$.
- $f(n) = \omega(g(n))$ and $g(n) = \omega(h(n))$ implies $f(n) = \omega(h(n))$.

Reflexivity

- $f(n) = \Theta(f(n))$
- $f(n) = O(f(n))$
- $f(n) = \Omega(f(n))$

Comparison of Functions

Symmetry

$$f(n) = \Theta(g(n)) \text{ iff } g(n) = \Theta(f(n))$$

Comparison of Functions

Symmetry

$$f(n) = \Theta(g(n)) \text{ iff } g(n) = \Theta(f(n))$$

Transpose Symmetry

- $f(n) = O(g(n))$ iff $g(n) = \Omega(f(n))$
- $f(n) = o(g(n))$ iff $g(n) = \omega(f(n))$

Comparison of Functions

Symmetry

$$f(n) = \Theta(g(n)) \text{ iff } g(n) = \Theta(f(n))$$

Transpose Symmetry

- $f(n) = O(g(n))$ iff $g(n) = \Omega(f(n))$
- $f(n) = o(g(n))$ iff $g(n) = \omega(f(n))$

Trichotomy

For any two real numbers a and b , exactly one of the following must hold: $a < b, a = b, a > b$.
However, this property does not carry over to asymptotic notation.